

PROVISIONAL PATENT APPLICATION

SYSTEM AND METHOD FOR PRIVACY-PRESERVING AND BLAST-RADIUS-BOUNDED EXECUTION OF AI-AGENT-AUTHORED ACTIONS THROUGH SAMPLE-DERIVED CODE VALIDATED IN A SANDBOX BEFORE PROMOTION TO A LIVE ENVIRONMENT

Authors: Tom Tang

Organization: Edge Vector Foundation

Date: June 4, 2026

Applicant

Edge Vector Foundation
14205 SE 36th Street, Suite 100, Bellevue, WA 98006

Inventors

- Tom Tang, Seattle, WA

Field of the Invention

This invention relates to the safe execution of actions authored by artificial intelligence agents on production systems holding sensitive data and credentials. More particularly, the invention provides an architecture in which an AI agent never observes live data, never holds secrets, and never invokes system tools directly. Instead, the agent receives a sample dataset that approximates the schema and selected statistical properties of a live dataset, authors a code module that performs the requested action, and the system executes that code module in a sandbox against the sample dataset to validate its behavior before promoting the same code module — by content-addressed identity — to execution against the live dataset in a production environment that resolves opaque secret handles within its own trusted boundary.

Cross-Reference to Related Applications

None (provisional filing).

Note: This application claims a workflow architecture for AI agents that is independent of, and complementary to, the inventions claimed in the co-pending provisional applications by the same applicant: “System and Method for Validating Safety and Correctness Properties of User-Submitted Pure Functions Executing Inside a Database Through Sandboxed Content-Addressed Modules” (which claims the verified-execution substrate as a registry of submitted code, including content-addressed identity and registration-time verification) and “System and Method for Controlling Access to Stored Data Through Composable Policy-Enforcing Interfaces with Conjunctive Multi-Mechanism Access Control and Failure Indistinguishable from Nonexistence” (which claims the data-access architecture). The present application claims the AI-agent workflow that may be implemented on top of, but does not require, those substrates: an AI agent’s interface to the system is *sample data plus code modules*, not live data or direct tool invocation, and the code module is validated against the sample before promotion to live execution.

Background of the Invention

Modern AI agents — large language models with tool-use, code-execution, or computer-use capabilities — perform actions on production systems by invoking tools, calling APIs, querying databases, and reading user data. This conventional architecture is structurally insecure in three concurrent dimensions:

1. **Data exposure to the agent.** To act on user data, an AI agent in current architectures must observe the user data. To send an email on a user’s behalf, it must read the user’s mailbox. To file an issue against a user’s records, it must see those records. There is no general mechanism by which an agent can decide what to do without first seeing the data it would decide about. Whatever the agent observes is incorporated into its context window, which may be logged, transmitted to model providers, used for training, or otherwise leak beyond the user’s control.
2. **Secret exposure to the agent.** To call APIs requiring authentication, agents are typically given live API keys, OAuth tokens, or other credentials. These credentials traverse the agent’s context window, the model provider’s infrastructure, and any logging or observability systems attached to the agent’s runtime. The agent’s system prompt, conversation history, or tool invocation traces become a credential-leak surface.
3. **Unbounded blast radius.** When an AI agent decides to act, it acts. The agent’s intent and the side-effecting invocation are architecturally collapsed: the act of “deciding to send an email” and the act of “sending an email” are the same act in agent runtimes built around direct tool invocation. A confused agent that decides to send twenty emails sends twenty emails. A jailbroken or prompt-injected agent that decides to drop a database drops the database. There is no general structural mechanism, separable from the agent’s own decision logic, that bounds the side-effects of agent actions.

Existing mitigations are partial:

- **Capability restrictions** (giving the agent only narrow tools) limit blast radius by enumerating allowed actions in advance, but require predefining every possible action and do not address the agent observing live data to make decisions.

- **Role-based or attribute-based access control** placed in front of agent tool calls can restrict which records the agent’s tools may operate on, but the access control is enforced only at the moment of the tool call — by which point the agent has already decided to call the tool, and the access decision provides no signal back to the agent’s decision process.
- **Differential privacy and synthetic data** address the data-observation surface but are typically applied to model training, not to runtime agent operation against a specific user’s data.
- **Code-generating agents** (assistants that produce source code rather than directly acting) shift the agent’s output from imperative tool calls to deterministic code, but the code they produce is typically reviewed by a human or executed against the live system without an automated validation step against representative data.
- **Sandboxed code execution** (e.g., container or virtualized runtimes) can isolate code at the execution layer, but does not address the agent’s prior observation of live data needed to write that code.
- **Constitutional and RLHF-based alignment** addresses agent behavior through training but provides no architectural separation: a misaligned agent in this regime still has the same direct access to data, secrets, and tools.

There is a need for a workflow architecture in which an AI agent can produce useful, validated actions on production data and systems without itself observing live data, without itself holding credentials, and without itself directly invoking side-effecting tools — and in which the agent’s authored actions are mechanically validated for correctness against representative data before any action is taken against the live system.

Summary of the Invention

The present invention provides a method, system, and computer-readable medium for safe and privacy-preserving execution of AI-agent-authored actions on a production system. The method is referred to herein as **sample-validated code promotion**. It comprises:

1. **Sample-mediated agent input.** When an AI agent receives a request that, under conventional architectures, would require observation of live data to decide an action, the system instead provides the agent with a *sample dataset* — a representation that preserves the schema and selected statistical properties of the live dataset but does not contain values from the live dataset. The agent reasons about the requested action using the sample.
2. **Code-as-action authorship.** The system does not authorize the agent to invoke side-effecting system tools directly. Instead, the system requires that the agent produce a *code module* — a deterministic bytecode artifact constrained to be a pure function with respect to declared inputs and declared side-effect channels — that, when executed against a dataset and a set of resolved side-effect channels, performs the requested action.
3. **Sandbox validation against the sample.** The agent-authored code module is executed in a sandboxed environment against the sample dataset. The system validates that the

code module’s behavior on the sample dataset satisfies declared correctness criteria. The criteria may include input/output examples provided by the agent alongside the code, declared invariants, declared bounds on side-effect channels (e.g., maximum number of recipients), and absence of disallowed external operations.

4. **Promotion to live execution by content-addressed identity.** Upon successful validation, the *same* code module — identified by a cryptographic hash of its bytecode — is executed in a production execution environment against the live dataset. The production environment computes the hash of the module it is about to execute and verifies that this hash matches the hash of the module that was validated; execution is refused on mismatch.
5. **Opaque secret references.** Secrets, credentials, API keys, and other privileged values used by the production execution environment are referenced by the agent-authored code module via opaque handles (e.g., `secret(‘stripe_api_key’)`). The agent never observes the underlying secret values; the production execution environment resolves the handle to the secret only at the moment of production execution and only within its own trusted boundary. Sandbox validation is performed without resolving the handle, or with the handle resolved to a non-functional placeholder, so the agent’s code is exercised on the sample without the sample environment requiring the real secret.
6. **Blast-radius constraints.** The agent declares, alongside the code module, the data records, system tools, external endpoints, and side-effect channels the code module is permitted to touch (e.g., “may call the email-sending tool with up to five recipients drawn from the contacts table”). The validator enforces that the agent’s declarations are consistent with the code module’s actual behavior on the sample. The production execution environment enforces the same declarations at runtime: any operation by the code module outside the declared bounds is structurally refused, regardless of the code module’s internal logic.
7. **Pure-function constraint with declared side effects.** The code module is constrained to be a pure function with respect to all of its inputs and outputs other than the declared side-effect channels. The code module cannot read system state, write system state, observe time, or invoke any operation other than (i) deterministic computation on its declared inputs and (ii) emission of values on declared side-effect channels. This constraint is enforced both by the sandbox and by the production execution environment.
8. **Differential validation (optional).** The system may execute the code module against the sample dataset both before and after a candidate live execution, comparing the post-execution sample-relative output against the validated pre-execution output. Divergence beyond a configurable threshold aborts the live execution or triggers a rollback.
9. **Human approval gate (optional).** Before promotion to live execution, the system may surface to a human operator: (i) the agent’s natural-language stated intent, (ii) a structured summary of the code module’s validated behavior on the sample, (iii) the declared side-effect bounds, and (iv) the content-addressed hash of the code module. Live execution proceeds only upon explicit operator approval. The operator does not need to inspect bytecode; the validation summary is sufficient.
10. **Append-only audit trail.** Each step of the workflow — the agent’s request, the sample dataset identifier, the agent’s authored code module hash, the agent’s declared side-effect bounds, the validation result, the human approval (if applicable), the production execution

result, and any rollback events — is recorded in an append-only log such that the chain of decisions leading to any production action is fully reconstructible.

Brief Description of the Drawings

Figure 1 depicts the three-boundary system architecture: the agent boundary (confining the AI agent to sample data), the validation sandbox (where the agent-authored code module executes against sample data before promotion), and the production boundary (where live data and secrets are held and the validated module executes).

Figure 2 depicts the promotion lifecycle: the sequential steps from sample-mediated agent input through code-module authorship, sandbox validation, content-addressed registration, and production execution, showing the feedback loop on validation failure and the append-only audit record at each step.

Figure 3 depicts the three-layer blast-radius bounding architecture: the agent’s declaration of side-effect bounds (Layer 1), the sandbox’s validation of declared bounds against sample-data behavior (Layer 2), and the production environment’s runtime enforcement of declared bounds independent of code-module logic (Layer 3).

Figure 4 depicts opaque secret handle resolution: a side-by-side comparison showing how the sandbox environment resolves a secret handle to a non-functional placeholder during validation, while the production execution environment resolves the same handle to the live credential value within its trusted boundary at the moment of production execution, confirming that the agent never observes the resolved secret in either environment.

Detailed Description of the Invention

Architecture Overview

A system embodying the invention comprises:

- An **agent runtime** that hosts the AI agent (e.g., a large language model with tool-use). The agent runtime exposes to the agent only sample-mediated inputs and the ability to author code modules; it does not expose live data, credentials, or direct invocation of side-effecting tools.
- A **sample generator** that, given a live dataset (or a description of it) and a sampling policy, produces a sample dataset preserving the schema and selected statistical properties but not the values of the live dataset.
- A **validation sandbox** that executes agent-authored code modules against sample datasets under a sandbox runtime that enforces the pure-function constraint and the declared side-effect bounds, and that compares observed behavior against agent-declared correctness criteria.

- A **production execution environment** that holds the live dataset, the trusted boundary in which secrets are resolved, and the side-effecting tool invocations. The production environment refuses to execute any code module whose hash does not match a code module that has been validated.
- An **append-only audit log** recording the decisions and observations at each step.

The agent has access to the agent runtime only. It does not directly access the sample generator’s source data, the validation sandbox’s internals, the production execution environment’s secrets, or the audit log’s storage.

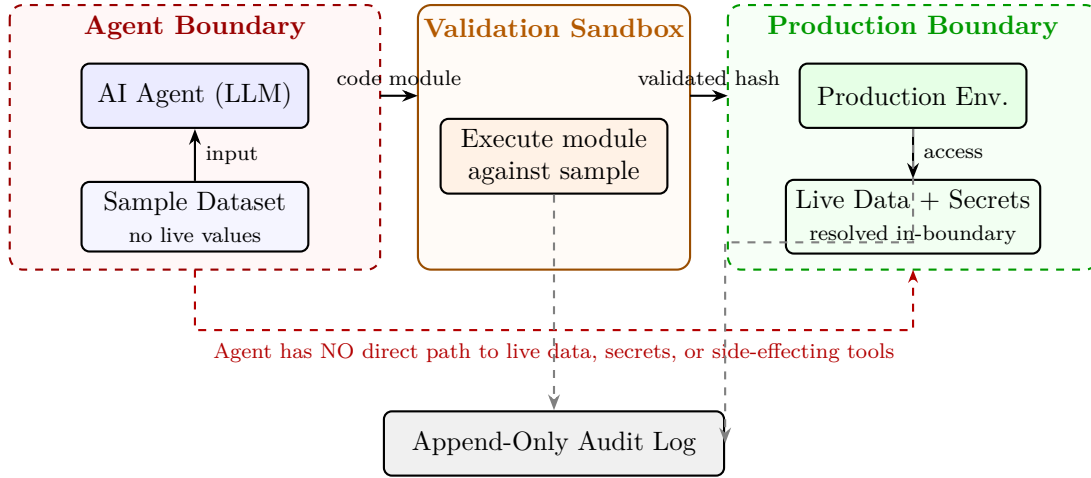


Figure 1: **FIG. 1** — System architecture. Three-boundary separation: the AI agent operates only within the agent boundary, receiving a sample dataset; the validation sandbox verifies code module behavior against the sample; the production boundary holds live data and secrets, admitting only the validated module’s cryptographic hash.

The Sample Dataset

The sample dataset preserves the schema of the live dataset such that a code module that compiles, executes, and produces well-formed output against the sample is structurally compatible with the live dataset. Beyond schema, the sample preserves the statistical properties needed for the agent to make a correct decision about what code to write: cardinality, value distributions, edge-case coverage, and the joint distribution of fields when correlations are decision-relevant. The sample does not preserve identifying values, sensitive field contents, or any record that maps one-to-one to a live record.

Sample generation strategies include, without limitation:

- **Synthetic generation** from a schema and a distribution model — values are drawn from distributions parameterized to match the live dataset’s aggregate properties, without any individual record being a live record.

- **De-identification of sampled records** — a small subset of live records are sampled, then de-identified by removing or generalizing identifying fields, perturbing numeric fields with calibrated noise, or substituting strings with vocabulary-equivalent placeholders.
- **Schema-matched substitution** — values from a public dataset whose schema matches the live dataset are substituted in, preserving structure without disclosing live values.

The choice of sampling strategy is policy-dependent and may be configured per dataset and per agent task. The invention does not require any particular sampling strategy; it requires only that the sample dataset preserve the schema and the statistical properties needed for the agent to author correct code, while not containing values from the live dataset.

Code-as-Action Authorship

The agent’s output, in response to a task that would conventionally require tool invocation, is a **code module** rather than a sequence of tool calls. The code module is a compiled bytecode artifact (e.g., WebAssembly, eBPF, JVM bytecode profile, or a content-addressable intermediate representation) constrained to be a pure function with respect to a typed input record (a row of the dataset) and a typed set of side-effect emissions.

The agent additionally declares, alongside the code module:

- The **input/output type signature** of the code module.
- The **declared side-effect channels** (e.g., “calls the email-sending tool”, “writes to the orders table”).
- The **bounds on each side-effect channel** (e.g., “at most five recipients per record”, “writes only records where status=’pending’”).
- The **correctness criteria** — a structured declaration of expected behavior, which may include input/output examples, post-conditions (“for every input row with field X, the output declares an email”), invariants (“no two outputs have the same recipient and same subject”), and natural-language summaries.
- The **secret handles** referenced by the code module (without secret values).

This declaration is produced by the agent as part of its output. It is the contract against which the validator and production environment enforce the agent’s intent.

Sandbox Validation

The validation sandbox executes the code module against the sample dataset and verifies the agent’s declarations:

1. **Type check.** The code module’s input type signature matches the schema of the sample dataset.

2. **Pure-function check.** The code module performs no operation outside (i) reading its declared inputs, (ii) deterministic computation, and (iii) emitting on declared side-effect channels. Attempts to read system state, observe time, open sockets, read files, or invoke undeclared operations are refused by the sandbox runtime.
3. **Side-effect bounds check.** The cumulative emissions on each declared channel, observed during sandbox execution, do not exceed the agent’s declared bounds.
4. **Correctness check.** The code module’s actual behavior on the sample dataset satisfies the agent-declared correctness criteria — input/output examples produce the declared outputs, post-conditions hold, invariants are maintained.
5. **Secret-handle resolution.** Where the code module references secrets via opaque handles, the validation sandbox resolves the handles to non-functional placeholders or to test-environment substitutes; the agent never observes the live secret value, and the validation does not require the live secret.
6. **Determinism check.** Re-executing the code module against the same sample dataset produces the same output, confirming the absence of hidden non-deterministic behavior.

If any check fails, the code module is rejected and the agent receives a structured failure report on which to author a revised module. If all checks pass, the code module is accepted, content-addressed by the cryptographic hash of its bytecode, and registered in a local, per-task validated-modules registry.

Promotion to Production

Promotion is performed by the production execution environment. When invoked with a code-module hash, the production environment:

1. Looks up the validated code module by hash.
2. Recomputes the hash of the bytecode it is about to execute and verifies a match.
3. Resolves declared secret handles to live secret values within its own trusted boundary; the secrets do not leave the production environment.
4. Executes the code module against the live dataset, with side-effect channels resolved to live tool invocations bound to the agent-declared and validator-approved bounds.
5. Records each side-effect emission and the final output in the append-only audit log.

If the hash does not match a validated module, or if any runtime check (pure-function constraint, side-effect bounds, declared types) fails, execution is refused and a failure record is written to the audit log. The agent does not observe the live dataset or the live secret values during this step; the agent’s contribution ended when it authored the code module.

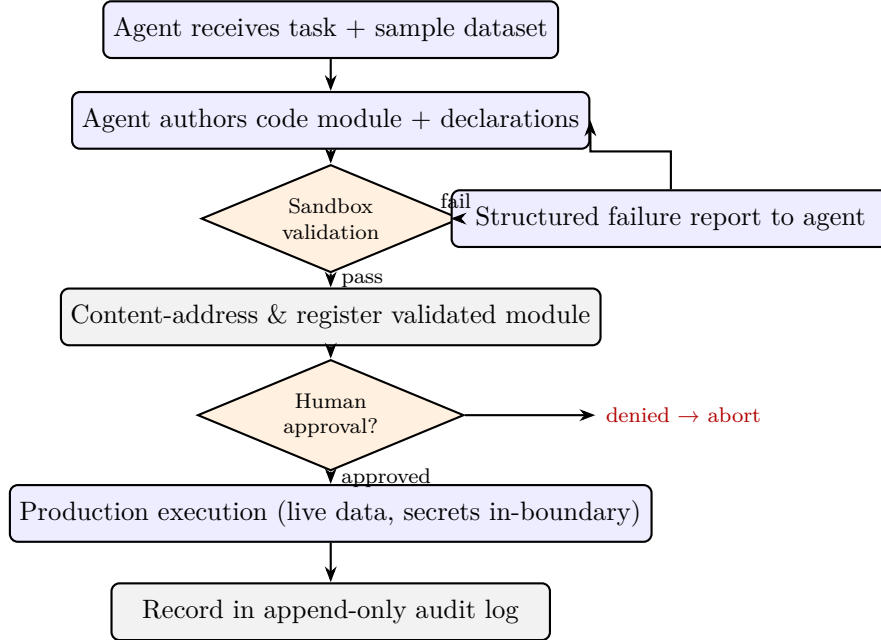


Figure 2: **FIG. 2** — Promotion lifecycle. Sequential steps from sample-mediated agent input through code-module authorship, sandbox validation, content-addressed registration, optional human approval, and production execution, showing the feedback loop on validation failure and the append-only audit record at each step.

Differential Validation and Rollback

Where the live dataset’s properties are sufficiently captured by the sample, the system may run the code module against the sample dataset both immediately before and immediately after the candidate live execution. Divergence between the pre- and post-execution sample-relative outputs, beyond a configurable threshold, indicates that the live execution had effects not captured in the sample-validated behavior — for example, a state-dependent side effect or an emergent interaction — and triggers an abort or rollback.

Rollback requires that the production execution environment can undo or compensate the code module’s emitted side effects. For idempotent or reversible side-effect channels (e.g., draft emails, transactional database writes, queueable API calls), this is straightforward; for irreversible channels (e.g., already-sent emails, externally observable state changes), rollback is best-effort and the system may, by configuration, refuse to proceed with code modules that touch irreversible channels without a higher-level human approval gate.

Embodiment: AI-Agent Workflow Built on a Compute-Without-Exposure Database Substrate

In one preferred embodiment, the production execution environment described above is realized by an existing data-infrastructure stack that already enforces compute-without-exposure properties for non-agent workloads. The substrate comprises: (i) a user-controlled database node holding the live dataset encrypted at rest with keys derived from a user-controlled secret; (ii) a sandboxed

bytecode runtime embedded in the database node that executes only content-addressed modules whose cryptographic hash matches a previously registered, registration-verified module; (iii) an encrypted communication relay that routes end-to-end encrypted requests between agent runtimes and the database node without decrypting payloads, validating policies, or executing modules; and (iv) a policy-enforcing interface layer over the live dataset that mediates every read and write through declared per-field policies (including any combination of trust distance, capability quotas, information-flow labels, and payment).

When the AI-agent workflow of the present invention is layered onto this substrate, the boundary between the agent and the live data is not a workflow-specific construction — it is the same boundary that the database substrate enforces under conventional, non-agent workloads. The agent’s interaction with the production environment reduces to (a) submitting a content-addressed code module’s hash to the sandbox via the encrypted relay, and (b) receiving the audit-log record of the resulting execution. The agent’s lack of access to live data, secrets, and direct tool invocation is structurally enforced by the substrate’s existing isolation properties; the workflow of the present invention reuses those properties as the boundary on which it builds. This embodiment is referred to herein as a “compute-without-exposure agent workflow.”

This embodiment composes the present invention with (a) the data-access architecture of the co-pending provisional application titled “System and Method for Controlling Access to Stored Data Through Composable Policy-Enforcing Interfaces with Conjunctive Multi-Mechanism Access Control and Failure Indistinguishable from Nonexistence,” which provides the policy-enforcing interface layer recited in component (iv); and (b) the validated-execution substrate of the co-pending provisional application titled “System and Method for Validating Safety and Correctness Properties of User-Submitted Pure Functions Executing Inside a Database Through Sandboxed Content-Addressed Modules,” which provides the sandboxed bytecode runtime and content-addressed module identity recited in component (ii). The independent claim of the present invention is not limited to this substrate; it is recited substrate-agnostically. The present subsection describes a concrete, technically specific reduction to practice.

Privacy and Blast-Radius Properties

The architecture provides the following structural guarantees, expressible as properties of the system rather than as requirements on agent behavior:

- **No live-data observation by the agent.** The agent’s input is the sample dataset. The live dataset is not provided to the agent, is not accessible to the agent, and is not represented in the agent’s context window in any form.
- **No secret observation by the agent.** Secrets are referenced by handle; the underlying values exist only inside the production execution environment’s trusted boundary.
- **Blast radius bounded by declaration.** The maximum side-effect impact of any agent action is bounded by the agent’s own declarations, validated against the sample by the sandbox, and enforced at runtime by the production environment. A malicious or confused agent cannot exceed the declared bounds because the production environment refuses unbounded operations regardless of code-module logic.

- **No direct tool invocation by the agent.** The agent’s only output is a code module plus declarations. Tool invocations occur within the production execution environment, bound to the validated code module’s hash and the validated declarations.
- **Reconstructible decision chain.** Every decision — sample identifier, code-module hash, validation outcome, human approval, production execution, rollback — is logged in an append-only audit, supporting post-hoc inspection and incident analysis.

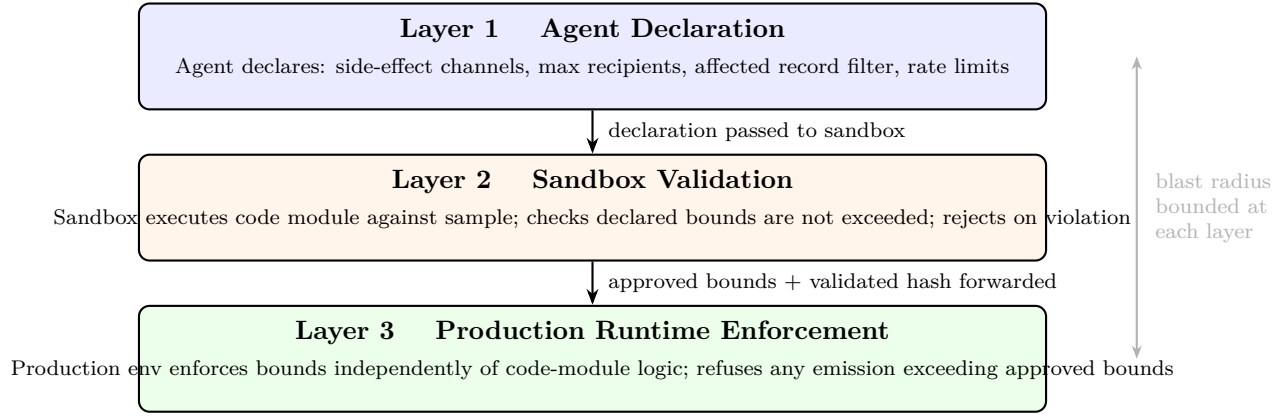


Figure 3: **FIG. 3** — Three-layer blast-radius bounding architecture. The agent’s declaration (Layer 1) specifies the maximum permissible side effects; the validation sandbox (Layer 2) verifies declared bounds against sample-data behavior; the production environment (Layer 3) enforces declared bounds at runtime independently of code-module logic, so a confused or adversarial agent cannot exceed its own declaration.

Opaque Secret Handle Resolution

An agent-authored code module references credentials via opaque handles rather than embedding actual values. During sandbox validation, handles resolve to non-functional placeholders, allowing the code module’s structure and behavior to be verified without requiring live credentials. In the production execution environment, handles resolve to actual credential values exclusively within the trusted boundary at the moment of execution. The agent never observes the resolved value in either context.

Claims

1. A computer-implemented method for safely executing actions authored by an artificial intelligence agent on a system holding live data, comprising:
 - (a) receiving, from an artificial intelligence agent, a request to perform an action on a live dataset managed by the system;
 - (b) providing the artificial intelligence agent with a sample dataset that preserves the schema and selected statistical properties of the live dataset and does not contain values from the live dataset;

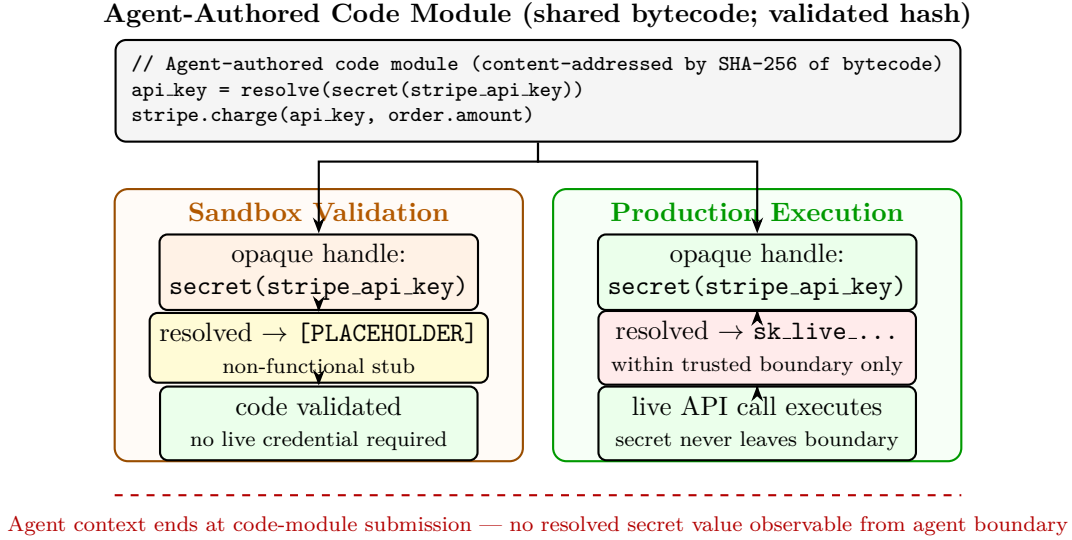


Figure 4: **FIG. 4** — Opaque secret handle resolution. The agent-authored code module references credentials via the opaque handle `secret(stripe_api_key)` without embedding the actual credential value. During sandbox validation (left), the handle resolves to a non-functional [PLACEHOLDER], allowing code-structure and behavior verification without the live credential. During production execution (right), the handle resolves to the live credential value exclusively within the production environment’s trusted boundary at the moment of execution. The agent’s context terminates at code-module submission; no resolved secret value is observable from the agent boundary.

- (c) receiving, from the artificial intelligence agent, a code module that, when executed against a dataset, performs the requested action;
 - (d) executing the code module in a sandboxed environment against the sample dataset, and validating that the code module’s behavior on the sample dataset satisfies declared correctness criteria;
 - (e) when validation in (d) succeeds, executing the same code module in a production execution environment against the live dataset; and
 - (f) wherein the artificial intelligence agent is not provided access to the live dataset, is not provided access to secrets used by the production execution environment, and does not directly invoke side-effecting tools of the system, throughout steps (a) through (e).
2. The method of claim 1, wherein the sample dataset is generated by one or more of: (i) synthetic generation that draws values from distributions parameterized to match aggregate statistical properties of the live dataset, (ii) de-identification of records sampled from the live dataset, or (iii) substitution of values from a schema-matched public dataset.
 3. The method of claim 1, wherein the code module is constrained to be a pure function with respect to declared inputs and a declared set of side-effect channels, and wherein the sandboxed environment in step (d) refuses any operation by the code module other than (i) reading its declared inputs, (ii) deterministic computation, and (iii) emission on declared side-effect channels.
 4. The method of claim 1, wherein the artificial intelligence agent provides, alongside the code module, a structured declaration of intended behavior comprising one or more of: (i) in-

put/output examples, (ii) post-conditions, (iii) invariants, and (iv) bounds on each declared side-effect channel; and wherein step (d) validates that the code module's actual behavior on the sample dataset satisfies the structured declaration.

5. The method of claim 1, wherein the code module is identified by a cryptographic hash of its bytecode, and wherein executing the same code module in step (e) comprises recomputing the cryptographic hash of the bytecode prior to execution and refusing execution unless the recomputed hash matches the hash of the code module validated in step (d).
6. The method of claim 1, wherein secrets used by the production execution environment, including credentials and API keys, are referenced by the code module via opaque handles, and wherein:
 - (a) the artificial intelligence agent never observes a value of any such secret;
 - (b) step (d) is performed without resolving the opaque handle, or with the opaque handle resolved to a non-functional placeholder; and
 - (c) step (e) resolves the opaque handle to the secret value only within the production execution environment's trusted boundary, at the moment of production execution.
7. The method of claim 1, wherein the artificial intelligence agent declares, alongside the code module, bounds on data records, system tools, external endpoints, and side-effect channels that the code module is permitted to touch, and wherein the production execution environment in step (e) refuses any operation by the code module that exceeds the declared bounds, regardless of the code module's internal logic.
8. The method of claim 1, further comprising executing the code module against the sample dataset both before and after a candidate live execution in step (e), comparing the post-execution sample-relative output against the pre-execution output, and aborting the live execution or triggering a rollback when divergence between the two exceeds a configurable threshold.
9. The method of claim 1, further comprising, before step (e), surfacing to a human operator (i) the artificial intelligence agent's stated intent, (ii) a structured summary of the code module's validated behavior on the sample dataset, (iii) the declared side-effect bounds, and (iv) the cryptographic hash of the code module; and proceeding to step (e) only upon explicit operator approval.
10. The method of claim 1, further comprising recording, in an append-only audit log, a record identifying: the artificial intelligence agent's request, the sample dataset identifier, the code module's cryptographic hash, the agent's declared correctness criteria and side-effect bounds, the validation result of step (d), any human approval, the production execution result of step (e), and any rollback event, such that the chain of decisions leading to any production action is fully reconstructible from the audit log.
11. The method of claim 3, wherein the pure-function constraint and the declared side-effect bounds are enforced both by the sandboxed environment of step (d) and by the production execution environment of step (e), such that the same constraint that admitted the code module in validation also bounds it in production.

12. The method of claim 1, wherein the artificial intelligence agent comprises a large language model and the code module is produced by the large language model in response to the request and the sample dataset.
13. The method of claim 1, wherein the production execution environment comprises:
 - (a) a user-controlled database node holding the live dataset encrypted at rest with keys derived from a user-controlled secret;
 - (b) a content-addressed sandboxed bytecode runtime, embedded within the user-controlled database node, that verifies a module's content-addressed cryptographic hash before execution and that constrains the executing module to no filesystem, network, clock, or system-call access beyond declared side-effect channels;
 - (c) an encrypted communication relay between the artificial intelligence agent's runtime and the user-controlled database node, configured to route end-to-end encrypted requests without decrypting or interpreting their content, and architecturally incapable of accessing plaintext data, of validating access policies, or of executing modules; and
 - (d) a policy-enforcing interface layer over the live dataset, through which all reads from and writes to the live dataset by the executing module are mediated, such that field-level access policies are evaluated at the moment of access independently of the executing module;

such that the boundary between the artificial intelligence agent and the live dataset and secrets is enforced by the same data-infrastructure components that enforce user-data access control under conventional non-agent workloads of the user-controlled database node.

14. A system comprising:
 - (a) one or more processors;
 - (b) an agent runtime configured to host an artificial intelligence agent and to expose to the agent only sample-mediated inputs and a facility to author code modules;
 - (c) a sample generator configured to produce, for a live dataset, a sample dataset that preserves the schema and selected statistical properties of the live dataset without containing values from the live dataset;
 - (d) a validation sandbox configured to execute agent-authored code modules against sample datasets under a runtime that enforces a pure-function constraint and declared side-effect bounds, and to validate the code modules' behavior against agent-declared correctness criteria;
 - (e) a production execution environment configured to execute, against a live dataset, only code modules whose cryptographic hash matches a code module validated by the validation sandbox, with secrets resolved within the production execution environment's trusted boundary; and
 - (f) memory storing instructions that, when executed by the one or more processors, cause the system to perform the method of any of claims 1–13.
15. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to perform the method of any of claims 1–13.

Abstract

A system and method for privacy-preserving and blast-radius-bounded execution of AI-agent-authored actions through sample-derived code validated in a sandbox before promotion to a live environment. An AI agent receives a sample dataset approximating the schema and selected statistical properties of a live dataset, rather than the live dataset itself, and authors a deterministic code module that performs the requested action, declares the side effects it intends to produce, and is validated in a sandbox against the sample data before being promoted—by content-addressed hash—to production execution. The agent never observes live data, never holds production credentials, and never directly invokes side-effecting tools; instead, opaque secret handles are resolved within the production environment’s trusted boundary. Sandbox validation verifies correctness against the sample data and confirms that the code module’s declared side effects are consistent with its behavior and within declared bounds before any live action is taken. The production environment refuses any code module whose declared bounds have not been validated in the sandbox and whose content-addressed identity does not match the validated module. The system provides privacy of live data and credentials from the agent, correctness validation of agent-authored actions against representative data, and structural blast-radius bounding enforced by the production environment regardless of agent intent, without requiring the agent to ever observe the live environment.

Drafting Notes (Internal — Not for Filing)

- **Distinguishing over capability-restricted agent tools.** Capability restrictions enumerate allowed actions in advance and apply at the moment of invocation. They do not address (i) the agent’s prior observation of live data needed to choose which capability to invoke, (ii) the agent’s need to hold credentials to authenticate the capability call, or (iii) the absence of an automated correctness-validation step against representative data before invocation. Claim 1 captures all three differentiators in a single workflow.
- **Distinguishing over code-generating assistants.** A code-generating assistant produces source code; what’s done with the code is downstream of the agent. The present invention claims the *workflow* in which (i) the agent’s input is sample data rather than live data, (ii) the agent’s output is constrained to a pure-function code module with declared side-effect bounds, (iii) the code module is mechanically validated against the sample, and (iv) the validated module is promoted by content-addressed identity. Each piece individually has analogues in prior art; the combination is the claim.
- **Distinguishing over sandboxed execution.** Container or VM sandboxes isolate at the execution layer but require the agent to have produced something to execute, typically with prior data observation. The present invention’s sandbox runs *agent-authored* code against *sample data* as a validation gate before live promotion. That use of the sandbox — as a per-task agent-output validator preceding production — is the novel application.
- **Distinguishing over differential privacy / synthetic data.** Synthetic data generation is an established technique. The novelty here is using a sample dataset *as the agent’s runtime input* during action authorship, not for training. Claim 2 captures the sampling techniques without claiming sampling itself.

- **Distinguishing over RLHF / Constitutional AI.** Behavioral training of agents is orthogonal to architectural separation. A perfectly aligned agent in current architectures still has direct access to live data, secrets, and tools; a misaligned agent under the present invention still cannot exceed the declared bounds because the production environment refuses unbounded operations regardless of agent state. Claims 6, 7, and 11 capture this.
- **The “code as action” framing in claim 1(c)–(d)** is doing significant work. Counsel may want to broaden by adding language covering the case where the agent emits a structured declarative artifact (e.g., a workflow specification, a query plan) that compiles to a code module — i.e., the claim should not be narrowable to imperative source code as the agent’s only valid output.
- **Claim 6 (secret opacity)** is independently strong as a privacy-of-credentials claim. Counsel may want to consider whether to promote it to a parallel independent claim covering “AI agent operations on systems requiring authentication, where the agent never observes credentials” — which would extend coverage to architectures that don’t have the full sample-validation flow but do have the secret-opacity guarantee.
- **Claim 7 (blast-radius declaration)** likewise has standalone value. An “agent declares its blast radius, system enforces, validator confirms consistency” pattern is independently patentable as a behavioral-bounding technique.
- **Cross-references to `patent_verified_function_execution`** (substrate that validates registered code) and **`patent_fold_access_architecture`** (substrate that gates data access) should be made explicit in the cross-reference section to disclaim overlap. The present application claims the *AI-agent workflow* — sample-mediated input, code-module output, validation-before-promotion — which is independent of and may be implemented on top of either substrate.
- **The terms “agent” and “AI agent”** in this application are intended to cover (i) large language models with tool-use, (ii) multi-agent systems, (iii) code-generating assistants operating in autonomous mode, and (iv) any other automated decision-making entity that authors actions on production systems. Counsel may want to add explicit coverage language in the spec to ensure narrowing arguments do not succeed in restricting “agent” to a particular model architecture.
- **Open question for counsel:** whether to file as a single provisional or split into two provisionals — one for the privacy/secret-opacity workflow (claims 1, 6, 12, 13, 14) and one for the validation/blast-radius workflow (claims 1, 3, 4, 5, 7, 8, 9, 11). A single filing is simpler; splitting may produce stronger independent claims if the secret-opacity guarantee turns out to be patentable independently of the sample-validation flow.

Prepared for provisional patent filing by Edge Vector Foundation.

Filing date: [TO BE DETERMINED]

This document constitutes a provisional patent application under 35 U.S.C. 111(b).